Task: Data quality focus

Introduction

This document presents a deliverable for internship task - DQ assessment for the E-Commerce dataset, which contains online retail transactions, including invoices, quantities, prices, customers, and countries.

The main aim is to define a set of meaningful, actionable DQ checks that would ensure this dataset is fit for downstream use (in analytics/reporting/BI context in general).

The presented approach follows DQ dimensions: Completeness, Validity, Uniqueness, Consistency, Accuracy, and Timeliness. Moreover, checks are organized into testing paths (Smoke, Critical, Extended) to reflect how they would be treated in a scenario of real production data pipeline. Severity labels are also added, as well as explanations what each checks verifies. In other words, a structured DQ framework is provided.

All provided examples use PostgreSQL and are written (to some extent) as pseudo-code, with aim to illustrate the validation logic. The checks were designed through manual inspection of the dataset structure and sample values, without loading the .csv file into a database. The focus of presented work is on the *design* and rationale of the checks, not solely on execution results.

Assumptions and context

With idea to make this examination realistic and somewhat aligned with the typical DQE workflow, I decided to „frame“ the dataset and the whole examination in the context of hypothetical, simplified data pipeline scenario shown below:

- Stage 1: Source system / CSV extract from E-Commerce platform
- Stage 2: Raw/staging layer – the data is stored in „as - is“ format
- Stage 3: DWH/Data warehouse layer – the data is cleaned, structured
- Stage 4: Reporting layer – the data is aggregated, trusted, ready for analytics/reporting consumption

In regard to structure of the „pipeline“, I decided to pretend I am assessing the table in Stage 3/DWH layer. Therefore, a scenario might be: I have a DWH-level table that should be clean enough for analytics, but also should not be trusted without further validation. In further text, the table will be referenced as dwh_ecommerce_data for better readability.

Therefore, following key assumptions have been made:

- The dataset represents a DWH table - dwh_ecommerce_data loaded from raw (or staging) layer
- A data model exists and it defines types, constraints, ranges and business rules. Since data model/mapping is not explicitly available, I am inferring reasonable expectations from the a) data b) domain context.
- Upstream, from this table, there is raw/staging layer which could be used to compare against for e.g. record count consistency
- Downstream, from this table, analytics/reporting will consume the data and main goal is to prevent bad data reaching them

Dataset overview

A dataset contains ~541 909 records of e-commerce transactions covering period from 2010-12-09 to 2011-12-09.

Column	**Inferred data type**	**Notes**
InvoiceNo	VARCHAR	Invoice identifier. Mostly 6-char strings.
StockCode	VARCHAR	Product code. Mostly 5-char strings, but includes non-standard codes – e.g. POST, M, AMAZONFEE, DOTCOM POSTAGE, gift_0001_30 etc. Length varies from 1 to 10+.
Description	VARCHAR	Text product description. Casing is inconsistent, varying from uppercase to sentence case format. Some records contains weird values/symbols e.g. ??, ?? missing, *Boombox Ipod Classic, ?sold as sets? etc. Presence of NULLs.
Quantity	INTEGER	Units per order. Has negative values – range is from -80995 to 81000. Some values seems unreasonably high.
InvoiceDate	TIMESTAMP	Transaction timestamp.
UnitPrice	NUMERIC	Price per unit. Range is from -11100 to 39000. Includes zero as price or values near to zero (e.g. 0,0001) records. Mixed precision is present, there are integers and decimals.
CustomerID	INTEGER	Customer identifier. NULLs are present.
Country	VARCHAR	Country. Stored in sentence case but includes ambiguous values e.g. Unspecified, European Community.

Please note that there is one important assumption regarding business rules. Namely, there are InvoiceNo values with prefix „C“. According to information provided on Kaggle link, these records seems to be connected with negative Quantity and UnitPrice value. This is treated as implicit business rule – not formally documented and have to be validated - meaning that these are canceled/returned orders.

Data Quality checklist

Prior to diving into specific DQ checks, checklist is made as a high-level map of *everything* worth validating, organised by testing paths:

- Smoke path: most crucial checks. If these fail, that should mean e.g. the data is not even available for further testing.

- Critical path: more like core integrity checks. Failures should e.g. directly compromise analytics accuracy. These must pass before data is promoted to the reporting layer.
- Extended path: more nuanced checks. They might reveal subtler issues and demonstrate DQ maturity, but a failure here would not necessarily block further data consumption, but e.g. it would generate a warning for further examination.

Therefore, checks are organised across these paths – prefixes are *S(moke)*, *C(critical)* and *E(xtended)* for better readability.

No	Name	What is being checked or verified	DQ dimension	Path	Approach	Severity	Expected outcome
1	S1	DWH schema and table dwh_ecommerce_data exist and are queriable	Completeness	Smoke	Positive	Critical	dwh schema exists, table (dwh.)dwh_ecommerce_data exists
2	S2	All expected columns are present with correct names	Validity	Smoke	Positive	Critical	All 8 columns exist and names are as expected (see: *Dataset overview*)
3	S3	dwh_ecommerce_data table is not empty	Completeness	Smoke	Positive	Critical	Basic check that data loading works (row count > 0)
4	C1	Record count matches between staging and DWH layers	Completeness	Critical	Positive	Critical	Count of DWH records should be equal to count of records in upstream layer or there is (documented) meaningful difference
5	C2	NULL values in columns	Completeness	Critical	Negative	Critical	No NULLs in all required columns
6	C3	No (full row) duplicates	Uniqueness	Critical	Negative	Critical	No rows that are identical across all required columns
7	C4	Data types match expected data model	Validity	Critical	Positive	Critical	All of required columns matches expected PostgreSQL data types
8	C5	Quantity and UnitPrice ranges are reasonable	Accuracy	Critical	Negative	Critical	No (unexplained) extreme values/outliers, negative values related to returns/cancellations in meaningful way
9	C6	Internal consistency across cancelled orders/transactions	Consistency	Critical	Negative	Critical	C-prefix invoices always have negative Quantity values, but non-C-prefix invoices never have negative Quantity values
10	C7	InvoiceDate within expected time-window, no future or past dates	Accuracy	Critical	Negative	Critical	Dates should cover period from 2010-12-09 to 2011-12-09, no date in past e.g. 1900. or dates in future

11	C8	Country values in line with a expected references	Validity	Critical	Negative	Warning	No values out of the list of expected countries
12	C9	UnitPrice precision is consistent	Validity	Critical	Positive	Warning	Prices are expressed via consistent 2-decimal format
13	E1	Mapping/relations between StockCode and Description columns is stable	Consistency	Extended	Negative	Warning	One StockCode value should be mapped/connected to one Description value
14	E2	Description column has no meaningless or placeholder values	Validity	Extended	Negative	Warning	No symbols, prefixes or unwanted values like ?, *, no suspiciously short entries
15	E3	StockCode format follows expected patterns	Validity	Extended	Negative	Warning	Non-standard codes (service fees etc.) should not be present but documented or logged and concerns should be raised

Data quality checks: detailed

Below are provided the specific detailed checks. They aim for thorough coverage across all DQ dimensions. Each check specifies - as required by the task - what it verifies, the DQ dimension, rationale behind the DQ checks, the SQL logic, and the severity level. As mentioned, these are illustrative pseudo-PostgreSQL-queries, in real production scenario, they would be embedded in a DQ framework.

Also, please note that column *Approach* refers to the approach to the testing, where *Positive* means it is needed to verify something expected is present; and in case of *Negative* it is needed to look for something that should not be there. Moreover, the DQ dimension Timeliness is not covered in the checks above because it can't be meaningfully verified against a static CSV file, since there are no pipeline metadata, load timestamps, or logs available to assess against. However, from a theoretical standpoint, timeliness checks could include - for example - verifying that MAX(InvoiceDate) falls within an expected window relative to the data loading date/timestamp, checking that no unexplained gaps larger than X business days exist between consecutive transaction dates, or confirming that upstream batch loads arrive within their expected schedule (e.g., daily at 05:00).

Detailed overview of each DQ check is presented below:

Check 1 (S1): DWH schema and table dwh_ecommerce_data exist and are queriable

DQ dimension	Path	Severity
Completeness	Smoke	Critical

Rationale behind the check

The DWH table dwh_ecommerce_data exists in the expected schema – let's say dwh schema - and can be queried. This is the most fundamental prerequisite, if the table does not exist, every other check is meaningless. This check might catch a failed migration, permissions change, schema rename etc.

PostgreSQL logic

```
SELECT COUNT(*) AS table_exists
FROM INFORMATION_SCHEMA.TABLES
WHERE TABLE_SCHEMA = 'dwh' AND TABLE_NAME = 'dwh_ecommerce_data';
```

Check 2 (S2): All expected columns are present with correct names

DQ dimension	Path	Severity
Validity	Smoke	Critical

Rationale behind the check

The table should contain all expected columns with correct names. This check could catch renamed columns, dropped columns, schema drift and similar. If a required column is missing, the queries referencing it will fail (meaning any pipeline step depending on it will break). The idea is to catch structural drift between what the data model specifies and what actually exists in the database/schema/table.

PostgreSQL logic

```
SELECT COLUMN_NAME
FROM INFORMATION_SCHEMA.COLUMNS
WHERE TABLE_NAME = 'dwh_ecommerce_data'
ORDER BY ORDINAL_POSITION;
```

Check 3 (S3): dwh_ecommerce_data table is not empty

DQ dimension	Path	Severity
Completeness	Smoke	Critical

Rationale behind the check

Check examine whether the table contains at least one row of data since table can exist, have the correct schema, and still be empty, meaning the load process either failed silently or ran against an empty source.

PostgreSQL logic

```
SELECT COUNT(*) AS total_rows
FROM dwh_ecommerce_data;
```

Check 4 (C1): Record count matches between staging and DWH layers

DQ dimension	Path	Severity
Completeness	Critical	Critical

Rationale behind the check

The number of records in the DWH table should match the number of rows delivered from the staging layer. Any discrepancy means data was silently lost or duplicated during the data movement. If the counts do not match and the difference is not documented, every downstream metric is potentially wrong.

PostgreSQL logic

```
SELECT COUNT(*)
FROM stg_ecommerce_data;

SELECT COUNT(*)
FROM dwh_ecommerce_data);
```

Check 5 (C2): NULL values in columns

DQ dimension	Path	Severity
Completeness	Critical	Critical

Rationale behind the check

NULL values in required columns mean the record cannot be aggregated, attributed, or validated. Since columns define what the transaction is, if any of them are NULL, the record is broken and should not reach the reporting layer. Description might be excluded here, while NULLs in it are a quality concern, a transaction without a description is still usable for analytics.

PostgreSQL logic

```
SELECT
    SUM(CASE WHEN InvoiceNo IS NULL THEN 1 ELSE 0 END) AS null_invoiceno,
    SUM(CASE WHEN StockCode IS NULL THEN 1 ELSE 0 END) AS null_stockcode,
    SUM(CASE WHEN Description IS NULL THEN 1 ELSE 0 END) AS null_description,
    SUM(CASE WHEN Quantity IS NULL THEN 1 ELSE 0 END) AS null_quantity,
```

```
    SUM(CASE WHEN InvoiceDate IS NULL THEN 1 ELSE 0 END) AS null_invoicedate,
    SUM(CASE WHEN CustomerID IS NULL THEN 1 ELSE 0 END) AS null_customerid,
    SUM(CASE WHEN UnitPrice IS NULL THEN 1 ELSE 0 END) AS null_unitprice
FROM dwh_ecommerce_data;
```

Check 6(C3): No (full row) duplicates

DQ dimension	Path	Severity
Uniqueness	Critical	Critical

Rationale behind the check

Check verifies if two rows in the DWH table are completely identical across all columns. Full-row duplicates might indicate a loading problem or a source extraction error. At the DWH layer, data should already be deduplicated. Check specifically aims for full-row duplicates specifically; partial duplicates may represent legitimate transactions.

PostgreSQL logic

```
SELECT InvoiceNo,
       StockCode,
       Description,
       Quantity,
       InvoiceDate,
       UnitPrice,
       CustomerID,
       Country,
       COUNT(*) AS duplicate_count
FROM dwh_ecommerce_data
GROUP BY InvoiceNo,
         StockCode,
         Description,
         Quantity,
         InvoiceDate,
         UnitPrice,
         CustomerID,
         Country
HAVING COUNT(*) > 1
ORDER BY duplicate_count DESC;
```

Check 7 (C4): Data types match expected data model

DQ dimension	Path	Severity
Validity	Critical	Critical

Rationale behind the check

The actual data types of all columns in the DWH table should match what the data model states. For example, Quantity should be integer, InvoiceDate should be timestamp, etc. At the DWH layer, data types must be enforced.

PostgreSQL logic

```
SELECT COLUMN_NAME,
       DATA_TYPE,
       IS_NULLABLE,
       CHARACTER_MAXIMUM_LENGTH,
       NUMERIC_PRECISION,
       NUMERIC_SCALE
FROM INFORMATION_SCHEMA.COLUMNS
WHERE TABLE_NAME = 'dwh_ecommerce_data'
ORDER BY ORDINAL_POSITION;
```

Check 8 (C5): Quantity and UnitPrice ranges are reasonable

DQ dimension	Path	Severity
Accuracy	Critical	Critical

Rationale behind the check

Numeric columns Quantity and UnitPrice should fall within business-reasonable bounds, meaning extremely large absolute values are likely errors or exceptional cases that need flagging. UnitPrices of exactly zero, or near-zero (0,001) are suspicious. Also, negative values should appear only on cancelled transactions (InvoiceNo starting with C). Therefore, negative values on regular invoices are a consistency violation. Since the dataset ranges from −80995 to 81000 for Quantity and −11000 to 39000 for UnitPrice this check is necessary. At the DWH layer, these outliers should be either corrected or discarded.

PostgreSQL logic

```
SELECT InvoiceNo,
       StockCode,
       Quantity,
       UnitPrice,
       (ABS(Quantity) > 10000) AS is_extreme_quantity,
       (Quantity = 0) AS is_zero_quantity,
       (UnitPrice = 0) AS is_zero_price,
       (UnitPrice > 0 AND UnitPrice < 0.01) AS is_near_zero_price,
       (ABS(UnitPrice) > 20000) AS is_extreme_price,
       (Quantity < 0 AND LEFT(InvoiceNo, 1) != 'C') AS
is_negative_quantity_not_cancelled,
       (UnitPrice < 0 AND LEFT(InvoiceNo, 1) != 'C') AS
is_negative_price_not_cancelled
FROM dwh_ecommerce_data
WHERE ABS(Quantity) > 10000
   OR Quantity = 0
   OR UnitPrice = 0
   OR (UnitPrice > 0 AND UnitPrice < 0.01)
   OR ABS(UnitPrice) > 20000
   OR (Quantity < 0 AND LEFT(InvoiceNo, 1) != 'C')
   OR (UnitPrice < 0 AND LEFT(InvoiceNo, 1) != 'C');
```

Check 9 (C6): Internal consistency across cancelled orders/transactions

DQ dimension	Path	Severity
Consistency	Critical	Critical

Rationale behind the check

There is an implicit business rule in this dataset which is accepted: invoices starting with "C" represent cancellations or returns, and they carry negative quantities. This check validates that the rule holds consistently in both directions. Therefore, all C-prefixed invoices should have negative Quantity, and all non-C invoices should have non-negative Quantity. If this rule is broken, it is not possible reliably separate sales from returns. Revenue calculations and order counts would all be wrong. Moreover, the rule is implicit, since it is not enforced by e.g. a database constraint.

PostgreSQL logic

```
SELECT InvoiceNo,
       StockCode,
       Quantity,
       UnitPrice
FROM dwh_ecommerce_data
WHERE LEFT(InvoiceNo, 1) = 'C'
  AND Quantity >= 0;

SELECT InvoiceNo,
       StockCode,
       Quantity,
       UnitPrice
FROM dwh_ecommerce_data
WHERE LEFT(InvoiceNo, 1) != 'C'
  AND Quantity < 0;
```

Check 10 (C7): InvoiceDate within expected and reasonable time-window, no future dates or (extreme) past dates

DQ dimension	Path	Severity
Accuracy	Critical	Critical

Rationale behind the check

All InvoiceDate values should fall within the known trading period (December 2010 to December 2011). There should be no dates before 2010-12-01 or dates after the expected data extraction cutoff, as well as dates in the extreme past/future. Out of range values can indicate parsing errors, timezone mishandling, or corrupted records. At the DWH layer, date integrity is non-negotiable because almost every analytical query filters/aggregates by time.

PostgreSQL logic

```
SELECT MIN(InvoiceDate) AS earliest_date,
       MAX(InvoiceDate) AS latest_date,
           SUM(CASE  WHEN  InvoiceDate  <  '2010-12-01'THEN  1  ELSE  0  END)  AS
before_expected_range,
          SUM(CASE  WHEN  InvoiceDate  >  CURRENT_DATE  THEN  1  ELSE  0  END)  AS
after_expected_range
FROM dwh_ecommerce_data;
```

Check 11 (C8): Country values in line with a expected references

DQ dimension	Path	Severity
Validity	Critical	Warning

Rationale behind the check

The Country column should contain recognisable, standardised country names. Values like "Unspecified" and "European Community" provide no geographic information. The dataset already lists individual EU member states (e.g. United Kingdom.), so these value are redundant and impossible to resolve to a single country. At the DWH layer, these should either be mapped to a real country (if possible from other context) or explicitly tagged somehow.

PostgreSQL logic

```
SELECT COUNT(*) AS invalid_country_count
FROM dwh_ecommerce_data
WHERE Country IN ('Unspecified', 'European Community')
      OR Country IS NULL;
```

Check 12 (C9): UnitPrice precision is consistent

DQ dimension	Path	Severity
Validity	Critical	Warning

Rationale behind the check

UnitPrice is expected to be a (monetary) value often with two decimal places (e.g., 2.55, 15.00). However, the dataset contains integers (15), values with more than two decimal places (0.001), and inconsistent formatting. At the DWH layer, numeric precision should be standardised with aim to avoid confusing results in reporting. For example a UnitPrices like 0.001 are also suspicious since it is too small to represent a real retail price and may indicate a placeholder, a test record, or a unit conversion error that happened upstream.

PostgreSQL logic

```
SELECT CASE
        WHEN UnitPrice != ROUND(UnitPrice, 2)  THEN 'more_than_2_decimals'
```

```
        WHEN UnitPrice = ROUND(UnitPrice, 0)    THEN 'integer_no_decimals'
        ELSE 'standard_2_decimals'
       END AS precision_type,
       COUNT(*) AS record_count
FROM dwh_ecommerce_data
GROUP BY precision_type;
```

Check 13 (E1): Mapping/relations between StockCode and Description columns is stable

DQ dimension	Path	Severity
Consistency	Extended	Warning

Rationale behind the check

A specific StockCode should consistently map to the same Description. If the same product code appears with multiple different descriptions, it should raise concerns. Some variations due to casing are cosmetic, but a StockCode mapping to semantically different product names is a real data integrity issue.

PostgreSQL logic

```
SELECT StockCode,
       COUNT(DISTINCT Description) AS description_variants,
       ARRAY_AGG(DISTINCT Description) AS descriptions
FROM dwh_ecommerce_data
WHERE Description IS NOT NULL
GROUP BY StockCode
HAVING COUNT(DISTINCT Description) > 1
ORDER BY description_variants DESC;
```

Check 14 (E2): Description column has no meaningless or placeholder values

DQ dimension	Path	Severity
Validity	Extended	Warning

Rationale behind the check

The Description column should contain meaningful product names. Values consisting only of punctuation (e.g. ??), starting with special characters (*Boombox Ipod Classic), or resembling internal notes (e.g. ?sold as sets?, ?? missing) are not valid records for DWH table. At the DWH layer, text columns should have already been cleaned or at least flagged during transformation from staging. If such values persist here, they will appear in product reports and analytics. This check identifies patterns that should have been discarded.

PostgreSQL logic

```
SELECT InvoiceNo,
       StockCode,
       Description,
       CASE
            WHEN Description ~ '^[?*!#@.,;\-\s]+$' THEN 'PUNCTUATION_ONLY'
            WHEN Description ~ '^\?' THEN 'STARTS_WITH_QUESTION_MARK'
            WHEN Description ~ '^\*' THEN 'STARTS_WITH_ASTERISK'
            WHEN LENGTH(TRIM(Description)) < 3 THEN 'TOO_SHORT'
       END AS description_flag
FROM dwh_ecommerce_data
WHERE Description ~ '^[?*!#@.,;\-\s]+$'
   OR Description ~ '^[\?\*]'
   OR LENGTH(TRIM(Description)) < 3;
```

Check 15 (E3): StockCode format follows expected patterns

DQ dimension	Path	Severity
Validity	Extended	Warning

Rationale behind the check

Most StockCode values follow a standard pattern: 5 numeric digits, optionally followed by 1–2 alphabetic characters. However, the dataset contains non-standard values e.g. POST, M, AMAZONFEE, DOTCOM POSTAGE, BANK CHARGES, and gift_0001_30. These likely represent service charges, shipping fees, etc., but not products. If downstream queries do not account for it, analyses could include various fees. This check categorises StockCodes, checking how much of them does not conform to expected format.

PostgreSQL logic

```
SELECT StockCode,
       COUNT(*) AS occurrences,
       CASE
            WHEN StockCode ~ '^\d{5}[A-Za-z]{0,2}$' THEN 'STANDARD_PRODUCT'
            WHEN StockCode ~ '^\d+$' THEN 'NUMERIC_NON_STANDARD'
            WHEN UPPER(StockCode) IN ('POST',
                                      'POSTAGE',
                                      'DOTCOM POSTAGE',
                                      'AMAZONFEE') THEN 'FEE'
            WHEN StockCode LIKE 'gift_%' THEN 'GIFT_CARD'
            ELSE 'OTHER_NON_STANDARD'
        END AS stockcode_category
FROM dwh_ecommerce_data
GROUP BY StockCode
ORDER BY stockcode_category, occurrences DESC;
```

Summary

All checks in this document were designed and presented as pseudo-SQL, as appropriate for the scope of this task. The underlying assumption is that in a production environment, however, these checks would be automated, scheduled, and integrated into the data pipeline. Moreover, they should be monitored over time and expanded as the data and its consumption evolve. Such approach might ensure visibility, pinpointing where the problems are and how severe they are.

Also, the presented approach might help to identify key DQ risks, for example:

Risk	**Impact**	**DQ Check**
Missing CustomerID in a significant number of orders/transactions	Customer-level analytics is problematic	Check 5 (C2)
Extreme outliers in Quantity and/or UnitPrice	Distortion of e.g. calculated totals, averages	Check 8 (C5)
Non-standard StockCodes mixed within product transactions	Fees (e.g. service, post) appears in product-level reports and dashboards	Check 15 (E3)
Non-standard values in Description field	Product-level reports contains meaningless entries	Check 14 (E2)
Ambiguous Country values	Geo-reports or segmentation of customers is problematic	Check 11 (C8)
Inconsistent UnitPrice precision	Errors in aggregations within reports	Check 12 (C9)

In general, DQ is an ongoing practice rather than a one-time activity. The checks presented in the document represent a illustrative baseline for how the dataset might be approached.